Evaluating Post-Quantum Key Exchange Algorithms in WireGuard

A Comparative Study of CRYSTALS-Kyber and HQC

Ludvig Järvi

Supervisor:
name lastname

Department of Computer Science, Electrical and Space Engineering
Luleå University of Technology
Luleå, Sweden

March 24, 2026

Acknowledgements

Abstract

Keywords: WireGuard, Post-Quantum, PQ, Post-Quantum Cryptography, PQC, ML-KEM, HQC,

Contents

1 Introduction **1**
1.1 Problem statement and research questions . . . 2
1.2 Scope and limitations . . . 3
1.3 The scarlet thread . . . 3
1.4 Thesis structure . . . 3

2 Background **4**
2.1 Asymmetric Cryptography . . . 4
2.1.1 RSA and Diffie-Hellman . . . 4
2.1.2 The Shift to Elliptic-Curve Cryptography . . . 5
2.1.3 Key Exchange vs. Key Encapsulation . . . 5
2.2 The Quantum Threat . . . 6
2.2.1 Quantum Computing . . . 6
2.2.2 Shor's Algorithm . . . 7
2.2.3 Grover's Algorithm . . . 7
2.2.4 "Harvest now, decrypt later." . . . 8
2.3 Post-Quantum Cryptography . . . 9
2.3.1 Lattice-Based Cryptography (CRYSTALS-Kyber) . . . 9
2.3.2 Code-Based Cryptography (Hamming Quasi-Cyclic) . . . 9
2.3.3 Hybridization Strategies and KEM Combiners . . . 9
2.4 Related work . . . 9
2.5 Concurrent work . . . 9

3 WireGuard **10**
3.1 Noise . . . 10
3.2 Handshake . . . 10
3.3 PQC Challenges . . . 10
3.4 Go implementation . . . 10
3.5 Motivation for Hybridization . . . 10

4 Method **11**
4.1 Benchmarking setup . . . 11
4.2 Program . . . 11

5 Results **12**

6 Discussion **13**
6.1 Active Research . . . 13

7 Conclusion **14**

Glossary

DH Diffie-Hellman.

HQC Hamming Quasi-Cyclic.

KEM Key-Encapsulation Mechanism.

ML-KEM Module-Lattice-Based Key-Encapsulation Mechanism.

NIST the National Institute of Standards and Technology.

PKE Public-Key Encryption.

PQ Post-Quantum.

PQC Post-Quantum Cryptography.

PSK Pre-Shared Key.

QKD Quantum Key Distribution.

VPN Virtual Private Network.

1 Introduction

What

In today's digital society, asymmetric encryption underpins nearly every aspect of global secure communication. Cryptographic protocols such as Transport Layer Security (TLS), Secure Shell (SSH), and Virtual Private Networks (VPNs) rely on asymmetric cryptography to securely establish shared secrets between previously untrusted parties. These cryptographic systems are based on the assumption that specific mathematical problems, notably integer factorization and discrete logarithms, are too hard for classical computers to solve efficiently.

However, this assumption may not hold in the long term. Rapid advances in quantum computing research suggest that large-scale, fault-tolerant quantum computers may become viable within the coming decades [1]. These are computers capable of leveraging the laws of quantum mechanics to perform quantum algorithms that achieves significant computational speedups for specific classes of problems in comparison to their classical counterparts. Such machines would be able to fundamentally break classical cryptography.

Two quantum algorithms stand out in this cryptographic context. Peter Shor's algorithm can efficiently factor large integers and compute discrete logarithms [2], and Lov Grover's algorithm [3], which provides a quadratic speedup to brute-force key searches. The former directly breaks the public-key primitives underlying RSA, Diffie-Hellman and elliptic-curve cryptography, while the latter reduces the effective security level of symmetric ciphers and hash functions by half.

Why

Once quantum computers reach this critical scale, they will likely be capable of decrypting nearly all historical and contemporary communications secured with current public-key systems. Moreover, adversaries can already capture encrypted traffic today with the intent of decrypting it later when quantum capabilities become available, a strategy commonly referred to as "harvest now, decrypt later." [4] This threat makes it essential to begin the transition toward quantum-resistant cryptography as soon as possible, since deploying new cryptographic standards across the Internet ecosystem will take many years.

To stay ahead of this threat, the cryptographic community is actively developing Post-Quantum Cryptography (PQC) algorithms. These are new types of "quantum-resistant" mechanisms designed to be secure against both today's computers and tomorrow's quantum machines. The algorithms are based on mathematical problems believed to remain hard for quantum computers, such as those derived from lattices, error-correcting codes, or hash functions. The National Institute of Standards and Technology (NIST) initiated, back in 2016, a global effort to identify and standardize these new methods [5], resulting in several promising candidates.

After several years of public evaluation, the NIST Post-Quantum Cryptography Standardization Process selected and standardized CRYSTALS-Kyber [6], a lattice-based Key-Encapsulation Mechanism (KEM), and is currently in the process of standardizing Hamming Quasi-Cyclic (HQC) [7], a code-based KEM, as next-generation standards for public-key encryption and key exchange. These schemes are now considered the primary building blocks for securing Internet communication in the quantum era.

How

While the theoretical security of these algorithms has been extensively studied, their practical deployment within real-world communication protocols remains an active research challenge. Post-quantum schemes often have larger key sizes, ciphertexts, and computational overhead compared to classical cryptographic methods. Such differences can significantly affect performance-sensitive protocols that prioritize efficiency.

One such protocol is WireGuard [8], a modern VPN protocol designed for minimalism, simplicity, and high throughput. WireGuard currently relies on elliptic-curve cryptography, specifically Curve25519, as part of its authenticated key exchange mechanism. While this design offers excellent efficiency on classical hardware, elliptic-curve cryptography is vulnerable to quantum attacks due to its reliance on the discrete logarithm problem.

Integrating PQC into WireGuard therefore presents several unique challenges related to efficiency, compatibility, and code maintainability. These include managing increased message sizes, maintaining low handshake latency, and preserving the protocol's minimalist design philosophy. Existing solutions, such as Rosenpass and PQ-WireGuard prototypes, have demonstrated the feasibility of quantum-resistant VPN connections through external or hybrid key exchanges [9][10].

Yet, few studies have systematically compared the performance characteristics of NIST-approved algorithms when integrated into WireGuard's architecture.

My contribution

This thesis addresses that gap by designing, implementing, and evaluating a hybrid WireGuard handshake that supports two NIST-selected Key-Encapsulation Mechanisms: CRYSTALS-Kyber and Hamming Quasi-Cyclic. The implementation enables an empirical comparison of their latency, computational overhead, memory usage, and message size, and contrasts the results against existing post-quantum WireGuard implementations. By quantifying these trade-offs, the work aims to provide practical insights into the real-world viability of post-quantum VPNs and contribute to the broader effort of securing Internet communication against future quantum threats.

1.1 Problem statement and research questions

How does different NIST-approved Post-Quantum Key-Encapsulation Mechanisms, specifically CRYSTALS-Kyber and HQC, affect the performance, resource usage, and practical deployability of a hybrid WireGuard handshake compared to existing post-quantum approaches such as Rosenpass?

RQ1. Evaluate whether the NIST selected PQC algorithms for key exchange and digital signatures are fit to use in the WireGuard VPN protocol without compromising its core design principles.

(a) Evaluate storage, bandwidth, and usability tradeoffs for these algorithms and ways to optimize them.

(b) Compare against previous implementations such as Rosenpass and other PQ-WireGuard prototypes.

(c) Find a balance between security level and ciphertext size.

RQ2. Explore the benefits and drawbacks of Hybridization, which combines classical cryptography with PQC.

1.2 Scope and limitations

Core scope:

- Userspace implementation in WireGuard-Go (no kernel changes)
- Evaluation on one primary hardware platform
- Comparative analysis and thesis documentation

Stretch goals:

- Cross-platform benchmarks (energy/performance scaling)
- Symbolic or formal modeling of the hybrid handshake
- Basic side-channel timing variance analysis

1.3 The scarlet thread

1.4 Thesis structure

This thesis is structured as:

2 Background

2.1 Asymmetric Cryptography

Asymmetric cryptography, also referred to as public-key cryptography, addresses the fundamental problem of symmetric cryptography in secure communication: how can two foreign parties establish a secure channel without first meeting up in person to exchange a secret key.

In order to solve this problem, asymmetric cryptography uses two mathematically related keys: a private key and a public key. The public key can be distributed freely, while the private key must remain secret. Messages encrypted with a public key can then only be decrypted using its corresponding private key, enabling secure communication without the need for a predetermined shared secret.

To illustrate this concept, imagine two people: Alice and Bob. Alice wants to send a secret message to Bob. To achieve this, they do the following:

1. Bob sends a box and an open padlock, to which only Bob has the key, to Alice.
2. Alice places her message in the box, locks it with the padlock, and sends it back to Bob.
3. Bob uses his key to unlock the box and is able to read Alice's message.

Even if some adversary were to intercept the box on its way back to Bob, they would be unable to open it and read Alice's message since they don't have Bob's key.

This simple analogy describes the core idea of asymmetric cryptography: separating the methods of encryption and decryption. Here the padlock represents the public key, and Bob's key the private key. He can send copies of his padlock to anyone who wants to send him a secret message, knowing only he can open them.

In practice, these systems use specific mathematical problems known as trapdoor functions [11]. These are problems that are easy to compute in one direction, but computationally infeasable to reverse without hidden information (the private key).

Asymmetric cryptography is not used directly to encrypt large amounts of data due to its computational overhead. Instead, it is commonly used during the initial handshake phase of a protocol to establish a shared symmetric key. Once this shared secret has been established, symmetric encryption algorithms can be used to protect the bulk of the communication efficiently.

2.1.1 RSA and Diffie-Hellman

The first generation of widely adopted asymmetric systems relied on two primary mathematical hurdles. The RSA (Rivest-Shamir-Adleman) algorithm, introduced in 1977 by Ron Rivest, Adi Shamir, and Leonard Adleman [12], built on the mathematical difficulty of factoring large composite integers. While multiplying two large prime numbers is computationally trivial, determining the original factors from their product is believed to be infeasible for sufficiently large numbers using classical algorithms. This asymmetry forms the basis of the RSA trapdoor function.

The other fundamental primitive introduced to public-key cryptography was the Diffie-Hellman (DH) key exchange, introduced in 1976 by Whitfield Diffie and Martin Hellman [13]. Rather than encrypting messages directly, Diffie-Hellman introduced the concept of the secure key exchange. Thus enabling two parties to securely establish a shared secret over an insecure channel. The security of this protocol relies on the hardness of the discrete logarithm problem, which involves recovering an exponent from a modular exponentiation operation.

These methods defined the first thirty years of digital security, but as computational power grew, the required key sizes for RSA and standard DH became unmanageable, leading to a need for more efficient mathematics.

2.1.2 The Shift to Elliptic-Curve Cryptography

In the early 2000s, many cryptograpic protocols began transitioning toward Elliptic-Curve Cryptography (ECC). Instead of operating over finite fields of integers, ECC relies on the algebraic structure of elliptic curves over finite fields. The advantage of the Elliptic Curve Discrete Logarithm Problem (ECDLP) is that it's significantly harder to solve per bit of key size than the two previous mathematical problems used in RSA and DH.

For example, a 256-bit elliptic curve key, such as Curve25519 used by WireGuard, provides a security level equivalent to a 3072-bit RSA or DH key [14]. This efficiency is one of the primary reasons modern protocols have adopted ECC. Smaller key sizes reduce bandwidth requirements and computational overhead, enabling faster handshakes and smaller packet headers, which are particularly important for performance-sensitive systems such as VPN protocols.

Despite their efficiency, elliptic-curve systems remain fundamentally vulnerable to quantum attacks due to their reliance on discrete logarithm problems. This vulnerability forms one of the key motivations for the development of post-quantum cryptographic algorithms.

Recognizing this risk, NIST has initiated a long-term transition toward quantum-resistant cryptographic algorithms. Current guidance recommends that organizations begin migrating away from classical public-key mechanisms in the coming years, with many traditional systems expected to be phased out after approximately 2030 as post-quantum standards are deployed [15]. This planned transition highlights the need to evaluate how existing protocols that rely heavily on elliptic-curve cryptography, such as WireGuard, can adapt to post-quantum cryptographic primitives.

2.1.3 Key Exchange vs. Key Encapsulation

It's important to distinguish between the two main methods for establishing shared cryptographic keys.

Traditional key exchange protocols, such as Diffie-Hellman and its elliptic-curve variant, employs an interactive process in which both parties contribute to the generation of a shared secret. Instead of sending the secret directly, each party exchanges intermediate values derived from their private key. Using these values, both participants can independently compute the same shared key.

In contrast, most post-quantum algorithms, including CRYSTALS-Kyber and HQC, are designed as Key Encapsulation Mechanisms (KEMs). Rather than performing a mutual

exchange of values, KEMs follow a sender-receiver model, where the sender generates a random symmetric key and encapsulates it using the recipient's public key. This process produces a ciphertext that is transmitted to the recipient. Using the corresponding private key, the recipient can decapsulate the ciphertext and recover the same shared symmetric key.

While the end result is the same, the logistical flow is different. This distinction is a core challenge when integrating post-quantum KEMs into existing protocols like WireGuard, which was originally designed for the interactive key exchange flow of Diffie-Hellman.

Despite this difference in communication structure, KEMs are generally preferred for post-quantum cryptography due to their simpler security models and resistance to several classes of implementation attacks.

2.2 The Quantum Threat

As discussed in the introduction, the long-term security of modern cryptographic systems relies on the assumption that certain mathematical problems are too computationally hard for classical computers to solve. However, the development of quantum computing challenges this assumption. Quantum computers operate according to the principles of quantum mechanics and are capable of executing algorithms that can outperform classical algorithms for specific types of problems.

There are two quantum algorithms that are especially relevant in this context: Shor's algorithm, which efficiently solves integer factorization and the discrete logarithm problem, and Grover's algorithm, which accelerates brute-force search. These algorithms are the frontrunners of the quantum threat to current cryptographic systems and are discussed in the following sections.

2.2.1 Quantum Computing

We must establish a high-level understanding of quantum computers work in order to understand how they are able to threaten current encryption.

A classical computer processes information using bits, binary values that exist in one of two states: 0 or 1. These bits are manipulated deterministically through electrical circuits made up of transistors and logic gates, which enables the computer to perform arithmetic and logical operations. At any given point in time, a classical system represents a single, well-defined state.

A quantum computer instead operates using quantum bits, commonly referred to as *qubits*, which has the ability to exist in a superposition of both states simultaneously. This means that a qubit represents a linear combination of the two states rather than a single deterministic value. When multiple qubits are combined, the system can represent a probability distribution of many possible states at once. For a system of n qubits, this corresponds to 2^n possible configurations, resulting in an exponentially large state space.

The other key property of quantum systems is entanglement, where the states of multiple qubits become correlated in a way such that the state of one qubit cannot be described independently of the others. Entanglement enables quantum computers to capture and exploit relationships between variables in ways that are not possible in classical systems.

Quantum algorithms leverage superposition, entanglement, and the manipulation of probability amplitudes to perform computations in a fundamentally different way than classical algorithms. By carefully controlling how probability amplitudes evolve during a computation, quantum algorithms can amplify the likelihood of correct solutions while cancelling out incorrect ones. This is what allows quantum computers to achieve significant computational speedups compared to their classical counterparts [16].

2.2.2 Shor's Algorithm

One of the most significant breakthroughs in quantum computing for cryptography was the discovery of Shor's algorithm by Peter Shor in 1994 [2]. The algorithm provides an efficient quantum method for solving the two mathematical problems that are central to modern public-key cryptography: integer factorization and the discrete logarithm problem.

In order to fully grasp the impact of Shor's algorithm, let's first consider the best known classical algorithm for solving these problems. The General Number Field Sieve (GNFS) which operates in sub-exponential time [17]. While it is significantly faster than naive approaches, the computational cost still grows at an infeasible rate at larger key sizes. The current largest RSA integer that has been factored using a software implementation of GNFS, with an optimized sieving step, called CADO-NFS is RSA-250 in February 2020 [18]. This number is 829 bits long and required approximately 2700 CPU core-years [19].

Shor's algorithm completely demolishes this notion of infeasability. By leveraging the principles of quantum mechanics, it is able to factor large integers and solve discrete logarithms in polynomial time. It accomplishes this by reducing the problem into a period-finding problem. Given an integer N, the algorithm selects a random value and constructs a function whose period is related to the factors of N. While finding such periods is computationally difficult for classical computers, quantum computers can solve this efficiently using the Quantum Fourier Transform (QFT).

The practical implications of Shor's algorithm have been extensively explored and optimized in later research. A widely cited paper by Craig Gidney and Martin Ekerå demonstrated in 2021 that factoring a 2048-bit RSA integer could be feasible using a quantum computer with around 20 million physical qubits and 8 hours of computation time, assuming realistic error rates, cycle times and reaction times [20]. This paper has recently recieved a follow-up by Gidney alone where he demonstrated that the same task could accomplished in less than a week using less than a million noisy qubits under the same assumed constraints [21].

The existence of Shor's algorithm therefore represents a fundamental threat to modern public-key cryptography. Since protocols such as TLS, SSH, and VPN systems rely on RSA, Diffie-Hellman, or elliptic-curve cryptography for secure key establishment, the availability of large-scale quantum computers would render these systems insecure.

2.2.3 Grover's Algorithm

Grover's algorithm [3] addresses a different class of problems than Shor's algorithm, specifically unstructured search. Given a search space of size N, a classical brute-force algorithm requires $\mathcal{O}(N)$ evaluations to find a target element. Grover's algorithm reduces this complexity to $\mathcal{O}(\sqrt{N})$ by leveraging quantum amplitude amplification.

In the context of cryptography, Grover's algorithm impacts symmetric-key primitives such as block ciphers and hash functions. But, unlike Shor's algorithm, Grover's algorithm does not completely break symmetric cryptographic schemes. Instead, it effectively halves their security level in terms of bit strength. This degradation can be mitigated by increasing key sizes. For example, a 128-bit symmetric key, which is considered secure against classical adversaries, would offer only 64 bits of security against a quantum adversary using Grover's algorithm. Consequently, doubling the key length (e.g., to 256 bits) is generally considered sufficient to maintain security in a post-quantum setting.

It is also important to note that practical implementations of Grover's algorithm face significant overhead due to error correction and circuit depth requirements, which further limits its near-term applicability. But it remains a critical consideration when evaluating the long-term security of symmetric cryptographic systems.

2.2.4 "Harvest now, decrypt later."

A common misconception about the quantum threat is that it only becomes relevant when sufficiently large quantum computers become available. This overlooks an already active risk model known as the "harvest now, decrypt later" (HNDL) [4].

In this risk model, an adversary intercepts and stores encrypted communication today with the intention of decrypting them in the future once sufficiently powerful quantum computers exist. This strategy is particularly concerning for data with long-term confidentiality requirements, such as government communications, intellectual property, personal data, and critical infrastructure information.

It is important to understand what part of modern encrypted communication is vulnerable in this context. While data at rest is often protected by symmetric encryption algorithms such as AES, which are considered more resistant to quantum attacks, the asymmetric public-key algorithms used during key exchange are the primary target. If an adversary can intercept the key exchange step of a session protected by TLS for example, they can store it and later use a quantum computer to recover the symmetric session key and decrypt the entire communication. The implication is that even data which appears to be well-protected at rest may be vulnerable if the key exchange that established its protection was recorded.

(Example of HNDL: Recent BGP hijackings)

The HNDL threat has driven significant regulatory action across multiple jurisdictions, each establishing strict timelines for the transition to post-quantum cryptography directly motivated by HNDL risk.

In the United States, the NSA's Commercial National Security Algorithm Suite 2.0 (CNSA 2.0) establishes a phased migration schedule for National Security Systems. Traditional networking equipment such as VPNs and routers is required to support and prefer CNSA 2.0 algorithms by 2026, and to use them exclusively by 2030. Large-scale public-key infrastructure systems, such as TLS, follow a slightly later schedule, with mandatory preference by 2030 and exclusive use by 2033 [22].

In Europe, the European Commission published a coordinated PQC implementation roadmap, which establishes a timeline for the transition to PQC for all member states. Under this roadmap, all member states are expected to have begun transitioning to

post-quantum cryptography by the end of 2026, with critical infrastructure systems fully transitioned to PQC no later than the end of 2030 [23].

At the standards level, NIST's deprecation timelines add further urgency. Classical quantum-vulnerable algorithms providing less than 128 bits of security are set to be deprecated after 2030 and disallowed after 2035, with algorithms above the 128-bit level set to follow the same eventual prohibition [24].

In summary, the "harvest now, decrypt later" threat establishes that the urgency of post-quantum migration is determined not by when quantum computers will arrive, but by the confidentiality requirements of data being transmitted and stored today. The combination of already-active data collection, long cryptographic migration timelines, and firm regulatory deadlines means that organizations handling sensitive data must treat the transition to quantum-resistant cryptography as an active and ongoing priority.

2.3 Post-Quantum Cryptography

Application context: how PQC fits into WireGuard-like VPNs. (Larger keys/ciphertexts)

Challenges

2.3.1 Lattice-Based Cryptography (CRYSTALS-Kyber)

CRYSTALS-Kyber [6], now officially renamed to ML-KEM in the NIST Post-Quantum Cryptography Standardization Process [25], is a

2.3.2 Code-Based Cryptography (Hamming Quasi-Cyclic)

Hamming Quasi-Cyclic (HQC) [7], is a

2.3.3 Hybridization Strategies and KEM Combiners

2.4 Related work

PQ-WireGuard 2021 [10]

Rosenpass 2024 [9]

PQ-WireGuard (Rebar) 2026 [26]

2.5 Concurrent work

NIST Post-Quantum Cryptography Standardization Process [5]

HQC standardization process

Regev's Algorithm [27] and discrete logarithms extension [28]

3 WireGuard

WireGuard is a modern VPN protocol developed by Jason A. Donenfeld [8].

Uses only UDP.

Widely deployed in the Linux kernel.

3.1 Noise

The Noise protocol framework is [29].

3.2 Handshake

3.3 PQC Challenges

Maximum Transmission Unit (1232 bytes for IPv6)

3.4 Go implementation

3.5 Motivation for Hybridization

4 Method

WireGuard-Go + liboqs

Use Level V parameters for all classification levels (NSA Commercial National Security Algorithm Suite 2.0).

4.1 Benchmarking setup

4.2 Program

5 Results

6 Discussion

As this field of study is constantly evolving

6.1 Active Research

Bring up current research

- HQC Standardization
- New KEMs and public key exchange methods

7 Conclusion

References

[1] M. Swayne. "Quantum Computing Roadmaps: A Look at the Maps and Predictions of Major Quantum Players." Accessed: 2026-03-10, The Quantum Insider. [Online]. Available: https://thequantuminsider.com/2025/05/16/quantum-computing-roadmaps-a-look-at-the-maps-and-predictions-of-major-quantum-players/

[2] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM Review*, vol. 41, no. 2, pp. 303–332, 1999. DOI: 10.1137/S0036144598347011 eprint: https://doi.org/10.1137/S0036144598347011. [Online]. Available: https://doi.org/10.1137/S0036144598347011

[3] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, ser. STOC '96, Philadelphia, Pennsylvania, USA: Association for Computing Machinery, 1996, pp. 212–219, ISBN: 0897917855. DOI: 10.1145/237814.237866 [Online]. Available: https://doi.org/10.1145/237814.237866

[4] J. Mascelli and M. Rodden, ""Harvest Now Decrypt Later": Examining Post-Quantum Cryptography and the Data Privacy Risks for Distributed Ledger Networks," Board of Governors of the Federal Reserve System, Washington, DC, Tech. Rep. 2025-093, 2025. DOI: 10.17016/FEDS.2025.093 [Online]. Available: https://doi.org/10.17016/FEDS.2025.093

[5] National Institute of Standards and Technology, *Post-Quantum Cryptography (PQC) Standardization Process*, https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization, [Online; accessed 19-February-2026], 2025.

[6] J. Bos et al., "CRYSTALS-Kyber: A CCA-Secure Module-Lattice-Based KEM," in *Proceedings of the 2018 IEEE European Symposium on Security and Privacy (EuroS&P)*, 2018, pp. 353–367. DOI: 10.1109/EuroSP.2018.00032

[7] C. Aguilar-Melchor et al., "Hamming Quasi-Cyclic (HQC) Specifications," HQC Team, Tech. Rep., 2025, Version dated 2025-08-22. [Online]. Available: https://pqc-hqc.org/doc/hqc_specifications_2025_08_22.pdf

[8] J. A. Donenfeld, "Wireguard: Next generation kernel network tunnel," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, The Internet Society, 2017. DOI: 10.14722/ndss.2017.23160 [Online]. Available: https://www.wireguard.com/papers/wireguard.pdf

[9] K. Varner, B. Lipp, L. Schmidt, M. Peischl, and W. Zaeske, "Rosenpass," 2024. [Online]. Available: https://rosenpass.eu/whitepaper.pdf

[10] A. Hülsing, K.-C. Ning, P. Schwabe, F. J. Weber, and P. R. Zimmermann, "Post-quantum wireguard," in *2021 IEEE Symposium on Security and Privacy (SP)*, 2021, pp. 304–321. DOI: 10.1109/SP40001.2021.00030

[11] A. C. Yao, "Theory and application of trapdoor functions," in *23rd Annual Symposium on Foundations of Computer Science (sfcs 1982)*, 1982, pp. 80–91. DOI: 10.1109/SFCS.1982.45

[12] R. L. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, 1978. DOI: 10.1145/359340.359342

[13] W. Diffie and M. Hellman, "New directions in cryptography," *IEEE Transactions on Information Theory*, vol. 22, no. 6, pp. 644–654, 1976. DOI: 10.1109/TIT.1976.1055638

[14] E. Barker, "Recommendation for Key Management: Part 1 – General," National Institute of Standards and Technology, Gaithersburg, MD, Tech. Rep. 800-57 Part 1 Rev. 5, May 2020. DOI: 10.6028/NIST.SP.800-57pt1r5 [Online]. Available: https://doi.org/10.6028/NIST.SP.800-57pt1r5

[15] D. Moody, R. Perlner, A. Regenscheid, A. Robinson, and D. Cooper, "Transition to Post-Quantum Cryptography Standards," National Institute of Standards and Technology, Gaithersburg, MD, Tech. Rep. NIST IR 8547 (Initial Public Draft), Nov. 2024. DOI: 10.6028/NIST.IR.8547.ipd [Online]. Available: https://doi.org/10.6028/NIST.IR.8547.ipd

[16] G. Brassard, P. Høyer, M. Mosca, and A. Tapp, "Quantum Amplitude Amplification and Estimation," *arXiv preprint arXiv:quant-ph/0005055*, May 2000. [Online]. Available: https://arxiv.org/abs/quant-ph/0005055

[17] J. P. Buhler, H. W. J. Lenstra, and C. Pomerance, "Factoring integers with the number field sieve," in *The Development of the Number Field Sieve*, A. K. Lenstra and H. W. J. Lenstra, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 1993, pp. 50–94, ISBN: 978-3-540-47892-8.

[18] The CADO-NFS Development Team, *CADO-NFS, An Implementation of the Number Field Sieve Algorithm*, https://cado-nfs.gitlabpages.inria.fr, Development version, 2024.

[19] F. Boudot, P. Gaudry, A. Guillevic, N. Heninger, E. Thomé, and P. Zimmermann, "Comparing the Difficulty of Factorization and Discrete Logarithm: A 240-Digit Experiment," in *Advances in Cryptology – CRYPTO 2020*, ser. Lecture Notes in Computer Science, vol. 12171, Cham: Springer, 2020, pp. 62–91. DOI: 10.1007/978-3-030-56784-2_3

[20] C. Gidney and M. Ekerå, "How to factor 2048-bit RSA integers in 8 hours using 20 million noisy qubits," *arXiv preprint arXiv:1905.09749*, Apr. 2021, Version 3. DOI: 10.22331/q-2021-04-15-433 [Online]. Available: https://arxiv.org/abs/1905.09749

[21] C. Gidney, "How to factor 2048-bit RSA integers with less than a million noisy qubits," *arXiv preprint arXiv:2505.15917*, May 2025. DOI: 10.48550/arXiv.2505.15917 [Online]. Available: https://arxiv.org/abs/2505.15917

[22] National Security Agency, "Announcing the Commercial National Security Algorithm Suite 2.0," National Security Agency, Tech. Rep. PP-22-1338, Sep. 2022, Cybersecurity Advisory, Version 1.0. [Online]. Available: https://media.defense.gov/2025/May/30/2003728741/-1/-1/0/CSA_CNSA_2.0_ALGORITHMS.PDF

[23] EU PQC Workstream, "A coordinated implementation roadmap for the transition to post-quantum cryptography," European Union, Tech. Rep. Part 1, Version 1.1, Jun. 2025, Published June 11, 2025. [Online]. Available: https://digital-strategy.ec.europa.eu/en/library/coordinated-implementation-roadmap-transition-post-quantum-cryptography

[24] D. Moody, *NIST PQC: The Road Ahead*, Presentation, Cryptographic Technology Group, Accessed: 2026-03-24, National Institute of Standards and Technology, Mar. 2025. [Online]. Available: https://csrc.nist.gov/csrc/media/Presentations/2025/nist-pqc-the-road-ahead/images-media/rwcpqc-march2025-moody.pdf

[25] NIST, "Module-Lattice-Based Key-Encapsulation Mechanism Standard," in *NIST, Tech. Rep.*, 2024. DOI: 10.6028/NIST.FIPS.203 [Online]. Available: https://doi.org/10.6028/NIST.FIPS.203

[26] K. Hashimoto, S. Katsumata, G. Niot, and T. Wiggers, "Revisiting PQ WireGuard: A comprehensive security analysis with a new design using Reinforced KEMs," in *IEEE Security & Privacy 2026*, 2026. [Online]. Available: https://thomwiggers.nl/publications/pq-wireguard/

[27] O. Regev, "An Efficient Quantum Factoring Algorithm," *arXiv preprint arXiv:2308.06572*, Aug. 2023. DOI: 10.48550/arXiv.2308.06572 [Online]. Available: https://arxiv.org/abs/2308.06572

[28] M. Ekerå and J. Gärtner, "Extending Regev's Factoring Algorithm to Compute Discrete Logarithms," *arXiv preprint*, Jan. 2024. [Online]. Available: https://arxiv.org/abs/2311.05545

[29] Trevor Perrin, *The Noise Protocol Framework*, https://noiseprotocol.org/noise.html, [Online; accessed 5-March-2026], 2018.